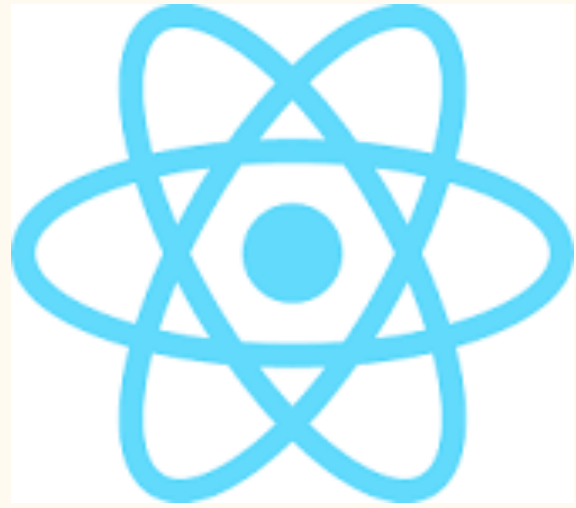


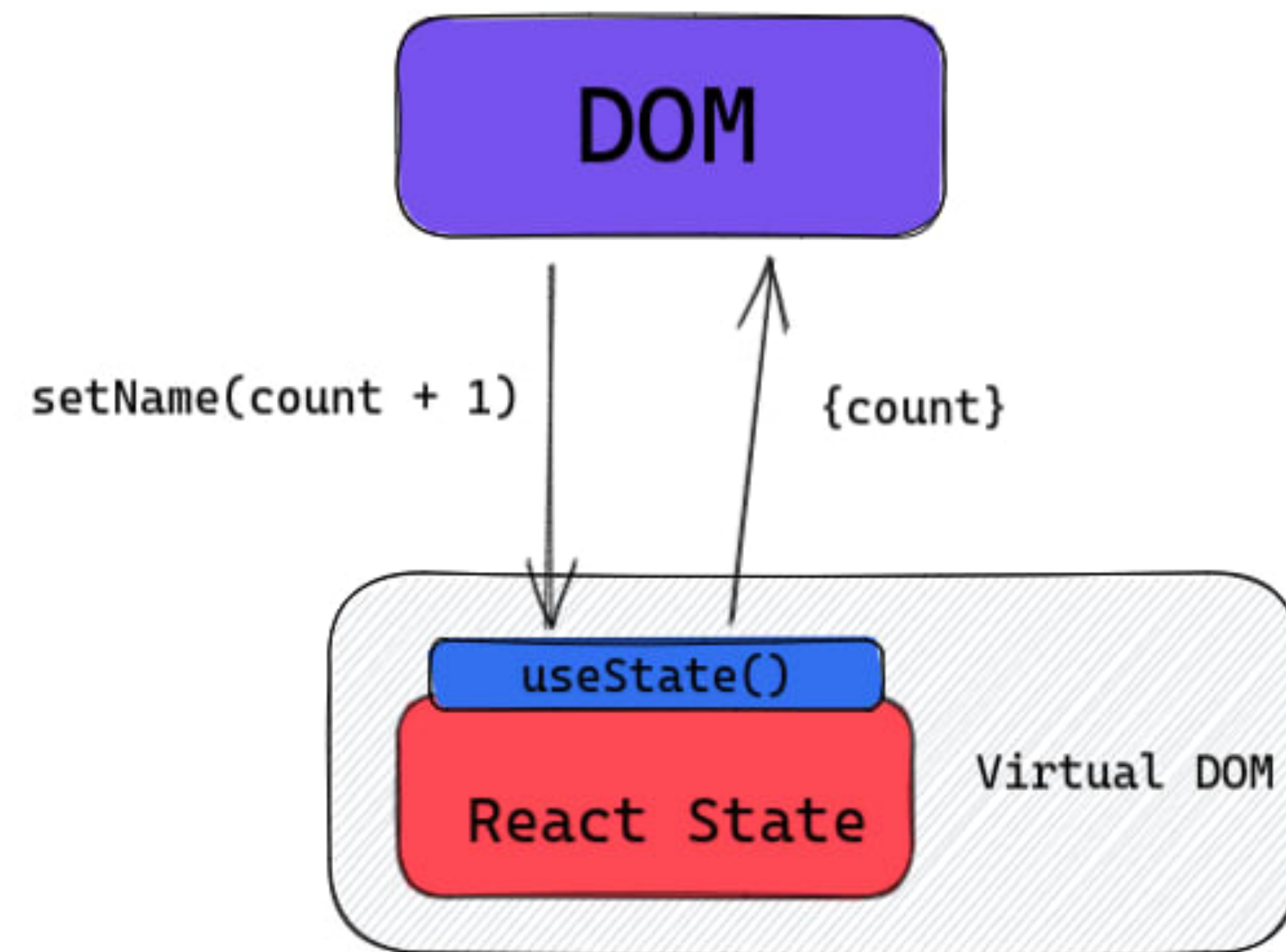
REACTIVITY & SIGNALS

I. Lesson



Reactivity in JS frameworks





REACT & LEPTOS

React = useState

Leptos = Signals

React vs Leptos

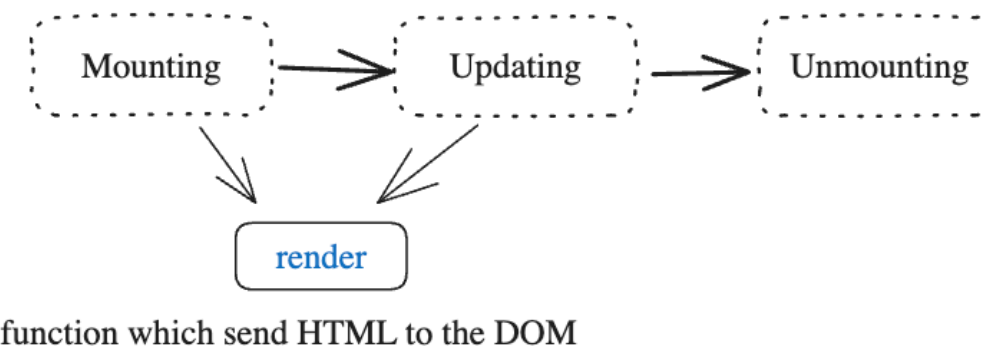
Topic	React <code>useState</code>	Leptos signals
Core idea	Component state + re-render	Fine-grained reactivity graph (signals/effects)
Update granularity	Re-run the component; VDOM diffs DOM	Update only the effects/nodes that read the signal
Dependency tracking	Manual (deps arrays in <code>useEffect</code> / <code>useMemo</code>)	Automatic: reading a signal registers a dependency
Rendering model	Virtual DOM reconciliation	Direct DOM updates (no VDOM)
Learning curve	Familiar to JS/TS devs	New reactive model + Rust ownership/borrowing
Effects & derived values	<code>useEffect</code> , <code>useMemo</code>	<code>Effect::new()</code> , <code>Memo::new()</code>
Performance implications	May need memoization to curb rerenders	Minimal work by default via fine-grained updates
State shape & type safety	TypeScript (structural typing), discipline against mutation	Rust types/ownership; explicit <code>get/set/update</code>

What is a Signal?

A **signal** in Leptos is a reactive value — a piece of state that **automatically notifies the system** when it changes, triggering **UI updates** or **recomputations** in any part of the application that depends on it.

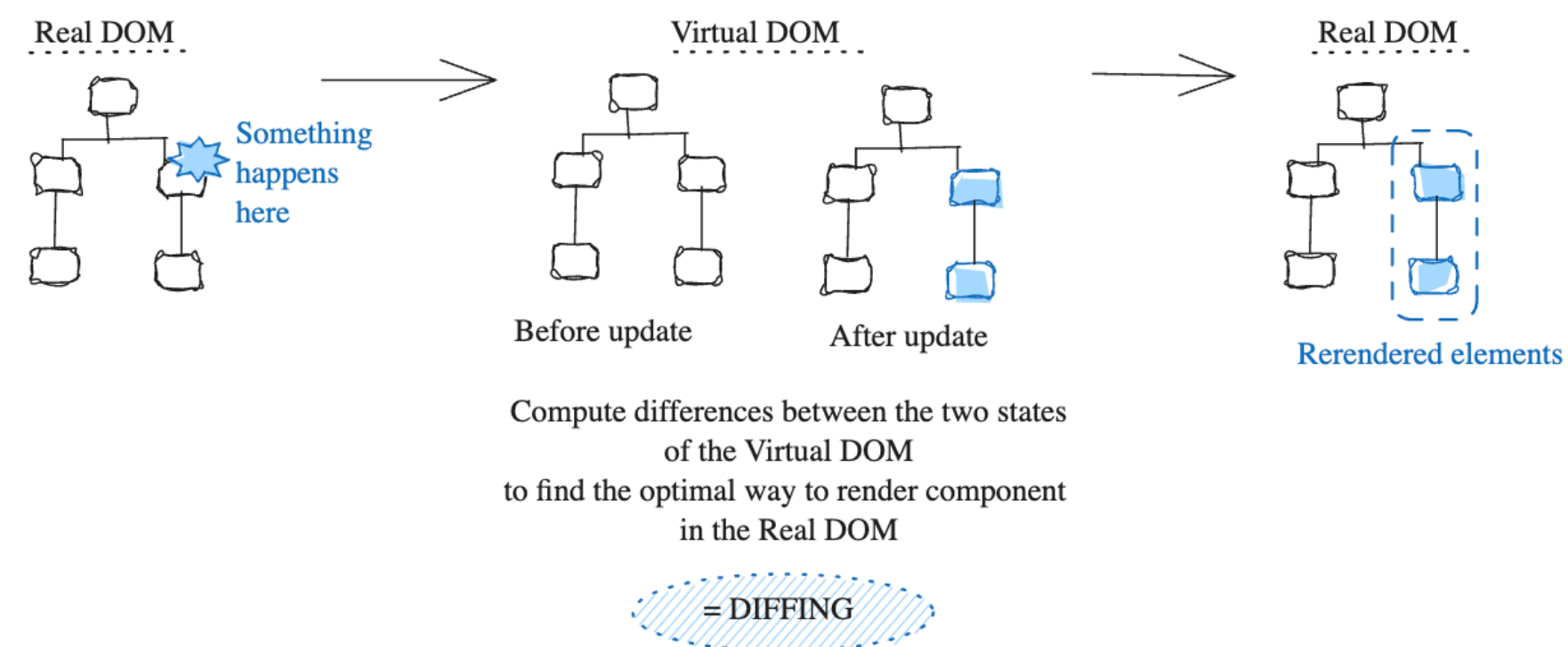
React JS

Component lifecycle:



Virtual DOM:

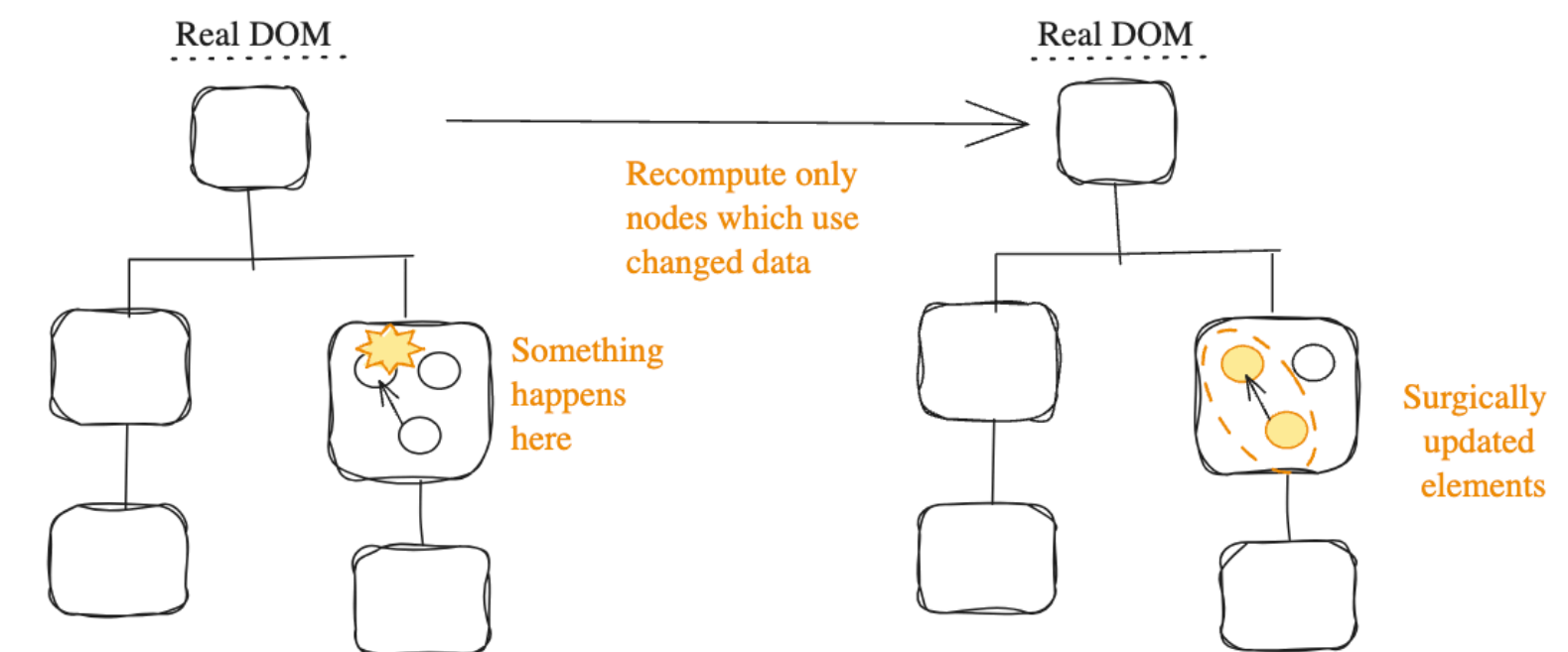
When a DOM element is rendered, all its children must be rendered too. The DOM is very long to be updated, hence it shouldn't be fully rendered. But it's difficult to compute directly the difference between two states of the DOM. That's why React uses a **Virtual DOM**



Solid JS

Fine-grained Reactivity:

Fine-grained Reactivity enables to get rid of Virtual DOM



Signals:

The key to this fine-grained reactivity is Signals are based on the Observer pattern A signal comes with a set of

Signals

Observer pattern subscribers

Code

```

[data, setData] = createSignal("xx")

read      write
myFunc = data()
setData("yy")
    
```

contrary to states in React, data is a method and not a variable

data changes recomputing its subscribers including myFunc

Under the hood

```

data.subscribers = []

data has no subscriber yet

data.subscribers = [myFunc]

add myFunc to subscribers of signal

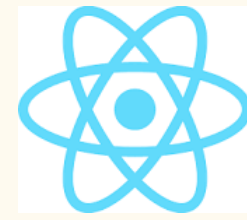
for subscriber in data.subscribers subscriber.recompute
    
```

<https://blog.theodo.com/2023/07/solidjs-beginner-virtual-dom-signals/>

It's like Leptos ported SolidJS to Rust



React



```
import React, { useState } from 'react';

function Counter() {
  // Declare a state variable called "count", with an initial value of 0
  const [count, setCount] = useState(0);

  // Function to increase the count
  const increment = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increase</button>
    </div>
  );
}

export default Counter;
```

Leptos



```
use leptos::prelude::*;

#[component]
pub fn Counter() -> impl IntoView {
  // Signal with initial value at 0
  let value = RwSignal::new(0);

  // Closure to increment the count
  let increment = move |_| value.update(|value| *value += 1);

  view! {
    <div>
      <span>"Value: " {value} "!"</span>
      <button on:click=increment>" + 1"</button>
    </div>
  }
}
```

TYPES OF SIGNALS



```
let (value, set_value) = signal(0);
```



```
let value = RwSignal::new(0);
```

GET AND SET SIGNALS

In Leptos, a **signal** holds reactive state and provides methods to **read** and **update** its value. You use `.get()` to **access** the current value of a signal, and `.set(new_value)` to **update** it.

When a signal is updated, any part of the UI or logic that depends on it will automatically re-run or re-render — enabling fine-grained reactivity with minimal boilerplate.

The move keyword in Leptos



```
let increment = move |_| set_value.update(|value| *value += 1);
```

II. Exercice

TRUNK & LEPTOS

trunk-rs/trunk

Build, bundle & ship your Rust WASM application to the web.



👤 121

Contributors

📦 200

Used by

💬 30

Discussions

★ 4k

Stars

🔗 284

Forks



Trunk is a **WASM** web application **bundler** for Rust. It automates tasks like **compiling Rust to WASM**, **bundling assets** (like CSS, JS, images), and serving the app locally with **hot-reloading**.

1. Install Trunk



```
cargo install --locked trunk
```

2. Install Leptos



```
rustup toolchain install nightly  
rustup target add wasm32-unknown-unknown  
rustup default nightly  
rustup update  
  
cargo install cargo-leptos
```



```
<!doctype html>
<html>
  <head>
    <link data-trunk rel="rust" data-wasm-opt="z" />
    <link data-trunk rel="icon" type="image/ico" href="/public/favicon.ico" />
  </head>
  <body></body>
</html>
```



```
[dependencies]
leptos = { version = "0.8", features = ["csr"] }
console_log = "1"
log = "0.4"
console_error_panic_hook = "0.1"
gloo-timers = { version = "0.3", features = ["futures"] }
web-sys = "0.3"
```

Simple Counter

```
use leptos::prelude::*;

mod simple_counter;

use simple_counter::SimpleCounter;

pub fn main() {
  _ = console_log::init_with_level(log::Level::Debug);
  console_error_panic_hook::set_once();

  mount_to_body(|| {
    view! { <SimpleCounter /> }
  })
}
```

SimpleCounter

```
use leptos::prelude::*;

#[component]
pub fn SimpleCounter() -> impl IntoView {
  let (value, set_value) = signal(0);

  let increment = move |_| set_value.update(|value| *value += 1);
  let decrement = move |_| set_value.update(|value| *value -= 1);
  let clear = move |_| set_value.set(0);

  view! {
    <div>
      <button on:click=clear>"Clear"</button>
      <button on:click=decrement>"-1"</button>
      <span>"Value: " {value} "!"</span>
      <button on:click=increment>" +1"</button>
    </div>
  }
}
```

Let's code

Your turn!